FreeRTOS Analysis

余嘉閔, 113511060

Abstract*—In this report, we analyze the context-switching mechanism of FreeRTOS, including the algorithms of xSemaphoreTake/Give *and* taskENTER/EXIT_CRITICAL. *We also measure the context-switching overhead under different conditions and evaluate how synchronization operations impact system performance.

***Keywords*—FreeRTOS, context switching, Aquila, synchronization**

I. Introduction

FreeRTOS is a real-time operating system widely used in embedded platforms, where efficient task scheduling and synchronization are essential. Because preemptive scheduling depends on timer interrupts and context switching, understanding its behavior directly affects system performance evaluation

This report analyzes the switching algorithm of FreeRTOS, measures context-switch overhead under different time-quantum settings, and examines how frequent preemption influences data-cache efficiency. It also compares the algorithms of xSemaphoreTake/Give and taskENTER/Leave_CRITICAL, and evaluates the synchronization overhead introduced by each mechanism.

II. Context Switching Algorithm

Trap can be classified into software trap(Exception) and hardware trap(Interrupt), both of them will jump into m mode, the following are the algorithm of the trap handler and the algorithm of context switching.

Trap can happen in multiple cases, for example, x divide by 0, but in the case of matrix multiplication task here, most trap are timer interrupt(interrupt) and yield(exception).

When a trap happens it jump into freertos_risc_v_trap_handler in portasm.s, secondly, it will store most reg and the context running now into stack(every time store, stack pointer need to +1) (for some cpu, having multiple sets of registers, it don't have to load and store these registers so frequently, instead, it only need to transfer to another set of register to accomplish these work) , and after it stores most common registers ,it will also store its mstatus, and store the current stack pointer into tcb.

In mpu(mcu having MMU and can run full os), the address in stack is not physically real address, but here, we don't have MMU, so the address in stack points to the real address in ram.

After storing the context running now, it now need to judge is it exception or interrupt this time.

If it is an interrupt, is it timer interrupt or external interrupt, the forward need to renew mtimecmp value in clint, and use xTaskIncrementTickto see whether it need to context switch(the function will see whether there exist another task waiting to be done). If context switch is needed, it will change sp to the stack address of the isr, and call vTaskSwitchContext.

If it is an exception, it needs to configure is it another exception or yielding this time. If it is yielding, it change sp to the stack address of the isr, and call vTaskSwitchContext.

What vTaskSwitchContext need to do is make pxCurrentTCB to be the task having the most priority now.(in the file tasks.c)

The final work is to load the context now need to run to the registers and mret to the mode of the task now needed to run.

III. taskENTER_CRITICAL/EXIT_CRITICAL Algorithm

The algorithm of taskENTER_CRITICAL can be found in tasks.c file the function: vTaskEnterCritical.

It disable interrupt, by calling portDISABLE_INTERRUPTS(the function disable interrupt by making the third bit of mstatus: MIE to be 0), and it later adding 1 to the critical section nesting layer(in order to support nesting conditions)

The algorithm of task EXIT_CRITICAL can be found in tasks.c file the function: vTaskExitCritical.

It make critical nesting counter -1 first, and see whether it becomes 0 now, if it is then it will enable interrupt. （MIE = 1）

IV. xSemaphoreTake/Give Algorithm

The algorithm of xSemaphoreTake/Give can be found in queue.c file, in the function xQueueSemaphoreTake and xQueueGenericSend.

xQueueSemaphoreTake() first enters a critical section and checks the current semaphore count (uxMessagesWaiting). If the count is greater than zero, meaning a "key" (token) is available, the function simply decrements the count and, checks whether any tasks are blocked waiting to send (i.e., tasks in xTasksWaitingToSend). If unblocking one of these tasks results in a higher-priority task becoming ready, the API requests a context switch, and take the original task to the ready list waiting for cpu to refetch the task. After exiting the critical section, the function returns success.

If the count is zero, the function examines xTicksToWait. When the wait time is zero, the function immediately exits with a failure status. If waut time is not zero now, blocking is allowed, the function records the timeout start time and prepares to wait.

Once outside the critical section, the function suspends the scheduler and locks the queue structure. If the timeout has not yet expired and the queue is still empty, the calling task must block. If the object is a mutex, priority inheritance may be

applied to raise the priority of the *current mutex holder* to prevent priority inversion. The calling task is then placed into the xTasksWaitingToReceive event list (it becomes Blocked, and if the semaphore is given back or the timeout expires, the *same blocked task* will be removed from the blocking event list and moved back into the Ready list.), and the scheduler is resumed; the CPU switches to another Ready task as determined by the scheduler.

If the timeout expires while the queue is still empty, the take operation fails. For mutexes, any previously applied priority inheritance is undone by lowering the holder's priority to the correct level. The function then returns failure.

If, at any time before or at timeout, another task or an ISR gives the semaphore and the queue becomes non-empty, the task will be unblocked and will re-enter the top of the loop, where it will retry the fast path and successfully obtain the semaphore.

For xSemaphoreGive, it first determines if the queue has space available (not full) or if overwrite mode is used, the kernel copies the item into the queue. It then checks whether any task is blocked waiting to receive; if so, the highest-priority blocked task is unblocked. After unblocking, the kernel determines whether a context switch is required—for example, in the case of a mutex, priority inheritance may have temporarily raised the current task's priority, and releasing the mutex can cause priority restoration, which also triggers a context switch. The rest of code and procedure are similar to those of xQueueSemaphoreTake.

V. ANALYSIS

1. Individual analysis of the default case:

For Size 128 matrices, 4 tasks, 100 time quantum case.

Time spent : 1209 ticks.

Timer interrupt: 1,193 times (442,482 cyc spent, 370 cyc/trap)

Yielding : 841 times (34,015 cyc spent, 40 cyc/ trap)

taskENTER_CRITICAL/EXIT_CRITICAL

: 4,372 cyc spent

xSemaphoreTake/Giveand: 9,924 cyc spent

we can see the following analysis:

a. Timer interrupt spends more cycles per trap than yielding because it need to do more things before jump to vTaskSwitchContext, for example, it needs to renew mtimecmp and call xTaskIncrementTick.

b. xSemaphoreTake/Giveand spends more cycles than taskENTER_CRITICAL/EXIT_CRITICAL, the reason is similar to (a), xSemaphoreTake/Giveand algorithm is obviously more complex than the other function.

2. Size of matrix and miss/hit relationship:

Size of matrix	128	32
Hit times	49,325,894	1,039,759
Miss times	2,229,232	75,463
Miss rate	4.33%	6.77%

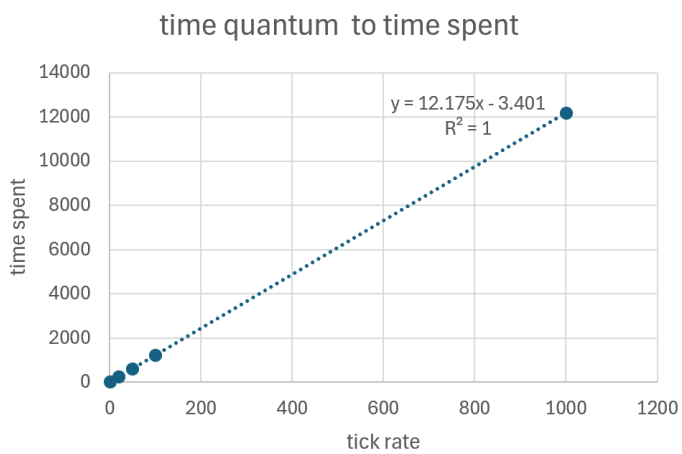
The difference of number of tasks and time quantum don't have much influence on hit and miss times.

Small amount of data have more miss rate because as the data becomes larger the same row/column will be called more times(spatial locality+ temporal locality).

As for why time quantum don't have much influence, maybe it actually don't context switch(every it enter the switch routine it ended up switch to itself not having to load a lot of reg like new ones)

3. Different time quantum

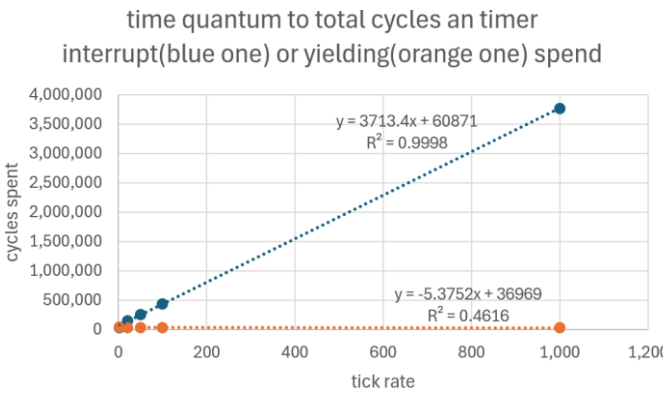
time quantum	**500ms**	**50ms**	**20ms**	**10ms**	**1ms**
time spent(tick)	24	242	605	1209	12172
timer interrupt time	**41**	**260**	**611**	**1,193**	**11,814**
cycles timer interrupt spend	**34,108**	**146,511**	**260,995**	**442,482**	**3,772,341**
yielding time	**953**	**830**	**715**	**841**	**660**
cycles yielding spend	**40,886**	**36,893**	**34,829**	**34,015**	**31,920**

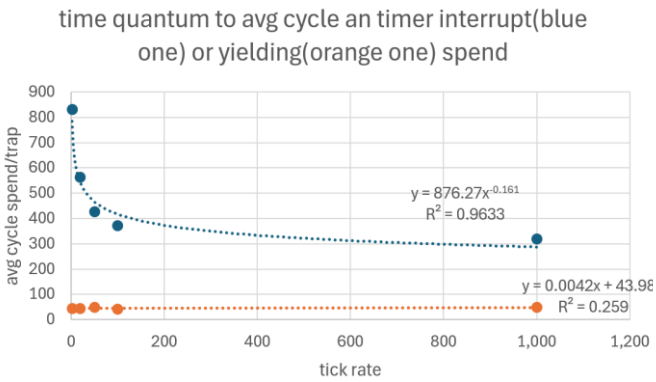


a. Tick rate is linearly proportional to the time spent.

Because when yielding, freertos didn't renew the timer interrupt(mtimecmp), so the rate timerinterrupt happens purely depends on tick rate, and also, because a cycle spent per timer interrupt is far larger than those

of exception, therefore, when time quantum decrease timer interrupt increases linearly, and time spent increases.





b. As time quantum increases, the cycles a timer interrupt spent decreases, this might because cache miss rate decreases(because these data are frequently used) and the effect of amortization.

4. Different Number of tasks

number of tasks	time spent	critical	semaphore
1	1209	4,372	9,924
4	1206	1,716	2,654
16	1223	15,990	35,091

The time spent don't have obvious differences in different number of tasks, but the cycles spent in synchronization functions are very different.(because the cycle of context switch is far more so the impact of thread number becomes trivial)

The reason why critical and semaphore related function spent more cycles in single thread than 4 thread might be that multi thread version can increase cache locality and context switch overhead becomes less for the matrix is spilt into smaller pieces of data.